

PhotonWeave: a Python-first CUDA finite-difference time-domain workbench for photonics and independent design ensembles

Hyoseok Park

Technical manuscript draft
19 September 2026

Abstract

PhotonWeave combines a serializable Python scene model, a browser-based geometry editor and a CUDA finite-difference time-domain solver. The implementation advances six electromagnetic field components on a Yee grid and supports convolutional perfectly matched layers, periodic and Bloch boundaries, passive Drude and Lorentz dispersion, and static graded rectilinear meshes. Time-domain point monitors and frequency-domain plane monitors share a consistent complex Fourier convention. Plane monitors account for Yee spatial staggering and the magnetic half time step before evaluating signed power-flux ratios against a matching reference. A process-based ensemble API evaluates independent structures concurrently and supplies a reproducible differential-evolution design loop. The formulation, software boundaries and numerical limitations are documented explicitly. A lossless dielectric slab provides an analytic validation of reflection, transmission and energy conservation. GPU throughput measurements include setup and data-transfer costs and distinguish one-device ensemble concurrency from single-simulation acceleration. The current implementation is a research workbench, not a feature-complete substitute for a commercial photonics platform.

1 Motivation and scope

Photonic simulation workflows combine geometry construction, broadband excitation, electromagnetic propagation and the extraction of observables. Repeating this sequence over many geometries is central to parameter sweeps and inverse design. A graphical editor lowers the cost of setting up a first model, while a programmable representation is essential for reproducibility and automation. PhotonWeave uses one validated scene model for both entry points. Every implemented native solver setting can be supplied without opening the user interface.

The implementation builds on the MIT-licensed `flaport/fdtd` package for its grid and backend foundation [1]. PhotonWeave supplies the derivative operators, boundary wrapping, CPML state, dispersive material update, mesh construction, monitor processing and execution orchestration described below. It uses NumPy for CPU calculations and PyTorch for CUDA arrays and graph capture [2, 3]. These dependencies and their authors are acknowledged rather than presented as new work.

TORCWA demonstrates the value of pairing an electromagnetic formulation with GPU execution and a programmable design workflow [4]. It solves a frequency-domain Fourier modal problem and provides automatic differentiation. PhotonWeave instead advances a time-domain Yee discretization. Its current design optimizer is a black-box method that evaluates independent forward simulations. No TORCWA code is incorporated and no equivalent adjoint or automatic-differentiation capability is claimed.

The current native scope includes two-dimensional z-invariant and three-dimensional models, four geometry families, isotropic dielectric and multipole dispersive materials, electric and magnetic vector soft sources, normal-incidence one-way planes and closed TFSF boxes, and point or planar frequency monitors. Automatic decay termination and empirical mesh convergence studies support validation of selected observables. Missing functions include general eigenmode ports, oblique and finite-aperture injection, conformal material integration, near-to-far transformation, tensor/nonlinear materials, and a

differentiable FDTD adjoint. A machine-readable feature inventory records these gaps separately from graphical and Python coverage and assigns priorities for the native photonics workflow.

2 Scene model and software architecture

The Project model contains a Region, a list of materials, structures, sources and monitors, and optional global source and monitor settings. Pydantic validation rejects unknown properties and non-finite values. Cross-field checks enforce grid dimensions, paired periodic boundaries, temporal Nyquist limits and source/monitor support within the non-PML region. Calling `Project.model_json_schema()` exposes the supported fields, enums and bounds.

The native model uses micrometres for geometry, wavelength and mesh spacing. Time is measured in seconds and frequency in hertz. Material oscillator parameters are angular frequencies. Bloch phase is specified in radians per unit-cell translation and carrier phase in degrees. An optional FDTD editing facade converts metre-based inputs to the same native project. These two length conventions are documented explicitly.

The browser edits a JSON representation of Project. The solver accepts the same representation and stores it with each NPZ result. Python export therefore represents the actual configured scene rather than a partial reconstruction of visible controls. Native simulation does not require a browser, a web server or any external simulation application.

Structure overlap is resolved before time stepping. A lower mesh-order integer takes priority, with the later object winning equal orders. Dielectric geometry that covers a dispersive region removes its oscillator ownership at the covered samples. The field image of permittivity represents the instantaneous relative permittivity, or ϵ_∞ for a dispersive material. It is not the full complex permittivity at a particular optical frequency.

2.1 Analytic solids and rigid coordinates

The solid model includes boxes, circular and elliptical cylinders, spheres and ellipsoids, full or partial elliptical rings, and extruded simple polygons. Polygon vertices define a local XY contour with a specified local z thickness. Both contour orientations and concavity are supported. Crossings, nonadjacent touching edges, adjacent overlapping edges and zero area are rejected. Polygon holes and general polyhedral surfaces remain outside this representation.

Let $\mathbf{c}$ denote the solid pivot and $R_a(\theta)$ a right-handed rotation about a fixed world axis. The local sample coordinate is

$$\mathbf{u} = R^T(\mathbf{x} - \mathbf{c}), \quad R = R_{a_3}(\theta_3)R_{a_2}(\theta_2)R_{a_1}(\theta_1)R_z(\theta_{\text{legacy}}). \quad (1)$$

Ellipsoid membership is $\sum_a (u_a/r_a)^2 \leq 1$. A cylinder uses the transverse ellipse inequality and $|u_z| \leq L_z/2$. A ring subtracts an inner ellipse and optionally restricts the physical polar angle $\text{atan2}(u_y, u_x)$. This is distinct from the ellipse parameter angle. A polygon uses an even-odd ray crossing test with explicit boundary inclusion. The two-dimensional solver samples the solid at $z = 0$ and then assumes invariant field physics along z.

For each enabled solid, material preparation computes a conservative axis-aligned support, locates its grid-index intervals by sorted search and evaluates membership only there. A small floating-point padding retains samples on support boundaries. Box and ellipsoid support functions and polygon vertex extrema give their bounds. Partial rings retain the bound of their complete elliptical cylinder. Overlap order and dispersive ownership are then applied to the retained samples. This changes host preparation work without changing the Yee grid, time step or electromagnetic updates outside the solids. Browser tessellation is used only for display and never determines material membership.

Independent tests compare rotation composition with a separate rotation implementation, analytic concave masks and volume convergence. The finest tested midpoint spacing gives less than 1.2 percent relative volume error for six rotated solids. This volume check does not establish electromagnetic convergence at their interfaces. Equivalent rotated box and polygon representations produce bitwise-identical complete optical outputs. CPU, CUDA and shared-cohort tests also cover mixed new solids.

The independent layout importer maps recognized primitive records to the same native solid model. Polygon storage uses global unrotated XY vertices and a separate pivot, which are converted to local contour coordinates. Ordered rotations and physical polar ring sectors are retained. An existing-object writer splices only decoded fields into the original byte stream. Variable-length names and vertex arrays may shift later records, whose old and new offsets are reported. The output is reparsed and its geometry and material response are checked before a file is returned. A no-op edit preserves all bytes. Author-generated fixtures verify untouched segments, transformed geometry and equivalent native CPU, CUDA and shared-cohort calculations. These checks establish neither general file compatibility nor agreement with a commercial electromagnetic solver. Saved mesh nodes remain fixed. A scene writer additionally handles recognized source temporal definitions and phases, monitor frequencies and windows, duration, CFL, and a subset of PML and periodic settings. It updates controlling input properties together with resolved values. External automatic sampling may choose a finer temporal stride, and the export report discloses this limit. Native global apodization is represented by effective local windows. Object-list changes, groups, new dispersive coefficients and general physical settings remain outside this subset.

3 Discrete electromagnetic formulation

3.1 Yee update and time convention

The six field components occupy staggered positions [5]. Electric components lie at the midpoint of their own coordinate direction and on nodes along the transverse directions. Magnetic components use the complementary dual positions. In a nonmagnetic dielectric, the reduced-field update has the form

$$E^{n+1} = E^n + S\epsilon_r^{-1}C^-(H^{n+1/2}) + s^{n+1}. \quad (2)$$

$$H^{n+3/2} = H^{n+1/2} - SC^+(E^{n+1}) + m^{n+3/2}. \quad (3)$$

Here $S = c\Delta t/h$, h is the smallest configured mesh step, and the curl operators contain dimensionless differences scaled by the appropriate metric. The soft electric source is added after the electric update and before the magnetic update. The implementation starts from zero fields. The first recorded electric sample is at Δt and the corresponding magnetic sample is at $3\Delta t/2$. This convention must be retained in Fourier transforms.

For d active spatial dimensions, let h_a denote the minimum primal width on axis a . The conservative time-step limit is

$$\Delta t_{\max} = \frac{\eta}{c\sqrt{\sum_{a=1}^d h_a^{-2}}}, \quad 0 < \eta \leq 0.99. \quad (4)$$

An explicit positive override may select $\Delta t \leq \Delta t_{\max}$. Equal axis spacings recover $\eta h/(c\sqrt{d})$. The background and dielectric indices are restricted to values at least one. The static graded grid retains this conservative fine-grid time step. Coarsening a background without changing the minimum axis spacings reduces the number of cells while retaining the physical duration at fixed step count. It does not implement local time stepping or temporal subcycling.

The two-dimensional case suppresses derivatives along z but retains all six vector components. An E_z electric excitation produces the usual TM_z subsystem. Bloch configurations use complex arrays. Nondispersive and ADE updates operate with the same real or complex field dtype.

3.2 Periodic and Bloch seams

Each periodic direction contains N independent cells. The code does not create a duplicated end plane. A Bloch field satisfies

$$F(x + L) = \exp(i\phi)F(x). \quad (5)$$

At a forward seam the difference uses $\exp(i\phi)F_0 - F_{N-1}$. At a backward seam it uses $F_0 - \exp(-i\phi)F_{N-1}$. The corresponding primal or dual spacing is applied to that difference. Source sheets can carry the same transverse Bloch phase. A point source in a periodic domain represents a periodically repeated source, which changes the physical problem from that of an isolated emitter.

3.3 Convolutional PML

Each absorbing face has independent layer count, σ scale, κ , α and polynomial exponents. The formulation follows convolutional PML [6]. Each transverse derivative in a PML segment owns a memory array ψ . The update is

$$\psi^{n+1} = b\psi^n + c_p D, \quad \tilde{D} = D/\kappa + \psi^{n+1}. \quad (6)$$

$$b = \exp[-(\sigma/\kappa + \alpha)S], \quad c_p = (b - 1)\sigma/(\sigma\kappa + \alpha\kappa^2). \quad (7)$$

The quotient is set to zero when its denominator vanishes. These are the code's dimensionless CPML coefficients, not a claim that similarly named controls in another solver have identical normalization. The depth profile, staggering and segment extents are explicit in `boundaries.py`. PML layers and the adjacent interior cell retain fine spacing on a graded grid.

Reflection depends on discretization, incidence, material continuation and profile parameters. The existence of a PML control does not establish a prescribed reflection tolerance. Relevant tests include periodic/Bloch identities, CPU/CUDA agreement and controlled boundary comparisons. General oblique, grazing and strongly dispersive PML convergence remains part of the validation roadmap.

4 Passive dispersive materials

The implemented frequency response uses the $\exp(-i\omega t)$ convention and up to 16 passive isotropic oscillators,

$$\epsilon(\omega) = \epsilon_\infty + \sum_{j=1}^K \frac{A_j}{\omega_{0j}^2 - \omega^2 - i\gamma_j\omega}. \quad (8)$$

For a Drude term, $\omega_0 = 0$ and $A = \omega_p^2$. For the single Lorentz input model, $A = \Delta\epsilon\omega_0^2$ and γ is twice the supplied linewidth. The multipole interface accepts ω_{0j} , A_j and γ_j directly in angular SI units. Positive oscillator strength and nonnegative damping enforce the intended passive parameter range. This is not a measured-data fitting system, and the bundled constant-index Si, SiN and SiO₂ entries are labelled approximations.

The solver evolves polarization P and the scaled current $Q = \Delta t \, dP/dt$ for each pole. Scaling avoids storing femtosecond-scale inverse-time magnitudes in float32 state. The oscillator equation and electric constitutive update are solved together by the trapezoidal rule. For a single pole, with old electric field E_{old} , provisional nondispersive field E_{free} , and current material state, define

$$d_0 = 1 + \gamma\Delta t/2 + (\omega_0\Delta t)^2/4. \quad (9)$$

$$a = (\omega_0\Delta t)^2/2, \quad k = A\Delta t^2/(4d_0), \quad r = (Q - aP)/d_0. \quad (10)$$

$$E_{\text{new}} = (\epsilon_\infty E_{\text{free}} - kE_{\text{old}} - r)/(\epsilon_\infty + k). \quad (11)$$

$$\begin{aligned} \Delta P &= r + k(E_{\text{new}} + E_{\text{old}}), \\ P_{\text{new}} &= P + \Delta P, \quad Q_{\text{new}} = 2\Delta P - Q. \end{aligned} \quad (12)$$

Auxiliary states are allocated only for the final samples owned by that material. With physical Yee sampling, ownership and state follow individual electric components. With legacy cell sampling, all three electric components share the same cell material. The constitutive discretization has the bilinear frequency substitution $\omega \mapsto (2/\Delta t) \tan(\omega\Delta t/2)$. This response alone does not describe spatial Yee dispersion or interface discretization error.

For multiple poles, the coefficients d_j , a_j , k_j and r_j are evaluated separately and the common electric field is obtained from

$$E_{\text{new}} = \frac{\epsilon_\infty E_{\text{free}} - (\sum_j k_j)E_{\text{old}} - \sum_j r_j}{\epsilon_\infty + \sum_j k_j}. \quad (13)$$

Each pole then uses this common field in Eq. 12. Applying single-pole electric corrections sequentially would not solve the coupled constitutive equation. The single-pole path retains its previous operation order. Passive fitting of sampled optical constants, anisotropy, magnetic dispersion, conductivity sheets, gain and nonlinear polarization remain absent.

5 Rectilinear and graded meshes

Uniform, graded and explicit node arrays are selectable. Uniform and graded generators accept independent minimum steps along the three axes. Explicit arrays must be finite, strictly increasing and centered on the native origin. Their endpoints define the domain, and geometry edits retain the arrays unchanged. All three representations use a tensor product. The graded algorithm projects geometry bounds, sources, monitor supports and refinement boxes into intervals on each axis. Refined intervals retain the corresponding fine lattice with two fine cells of padding. Cyclic seam cells and the PML support also remain fine.

Within an unrefined interval, a target spacing grows approximately linearly with distance from a refined edge, until limited by the user maximum and a background-wavelength cap. The node positions are equidistributed in the integral of the reciprocal target spacing,

$$u(x) = \int_{x_a}^x \frac{d\xi}{h_{\text{target}}(\xi)}, \quad u(x_j) = j \, u(x_b)/N. \quad (14)$$

The gap constructor checks the minimum spacing, maximum spacing and adjacent-cell grading ratio. A short or incompatible gap falls back to uniform cells. It does not create a remainder cell smaller than the configured fine step. The default neighboring-cell ratio is 1.25 and the default background target is 24 cells per configured wavelength.

Forward derivatives use primal cell widths $h_i = x_{i+1} - x_i$. Backward derivatives use dual distances $(h_i + h_{i-1})/2$. Metric factors h/h_i or their dual equivalents multiply the dimensionless differences, including periodic seams. An axis whose widths equal the reference step h bypasses metric multiplication. This construction avoids hanging nodes, but a small refinement box refines coordinate planes across the entire simulation. It can save substantially fewer cells than a spatially local subgrid would.

Explicit and independent-axis grids evaluate CPML depth using physical node positions and the actual absorbing-layer extent. Source corrections use the local normal-axis spacing. One-way planes require a constant normal step across the injection neighborhood. Closed TFSF boxes require constant spacing along each axis throughout the box and adjacent cells, while allowing different spacings between axes. General nonuniform incident-line propagation is not implemented. Material updates and Fourier phases use the actual configured time step.

Independent tests initialize rectangular-grid Fourier waves from the discrete dispersion relation in three dimensions. Irregular periodic and Bloch grids also satisfy a weighted curl-adjoint identity. With electric weights formed from one primal and two dual widths, and the complementary magnetic weights, the vacuum leapfrog invariant is

$$\mathcal{E}^n = \|E^n\|_{W_E}^2 + \|H^{n+1/2}\|_{W_H}^2 + S \operatorname{Re}\langle E^n, C^- H^{n+1/2} \rangle_{W_E}. \quad (15)$$

The numerical test preserves this quantity within 5×10^{-13} relative over 240 updates on independently generated positive node widths. This test excludes sources, materials and absorbing boundaries. Separate source-line and variable-depth PML pulse tests cover those stated configurations. Stability and these identities do not establish a general spatial accuracy bound.

Material assignment can sample either common cell centres or the separate physical Yee positions. The latter usually provides a more faithful interpretation of component placement, but it is still a staircase interface. No conformal integration or subpixel averaging is present. A coarser background mesh is a resolution heuristic, not an a posteriori error estimator. Metal skin depth, small gaps and resonant near fields require explicit convergence studies.

5.1 Empirical mesh convergence

The Python `mesh_convergence` routine rebuilds the same physical scene at decreasing spacings. It preserves the actual domain spans and PML thickness, rejecting spacings that cannot represent them as integer cell counts. The number of steps changes to preserve the original physical duration with less than one timestep of rounding. For a matched reference, the graded refinement intervals are frozen from the sample. Automatic decay termination is disabled during the study to keep sample and reference integration times identical.

The user supplies a finite real observable, such as a reference-normalized transmission spectrum on a fixed frequency grid. Success requires the final configured number of consecutive comparisons to satisfy the stated absolute and relative tolerances at every returned value. A cancelled study or invalid observable does not report convergence. Agreement of successive meshes is an empirical test for the chosen observable, not a general error bound or an adaptive mesh algorithm.

6 Termination and state diagnostics

An optional decay rule monitors the complete electric and magnetic fields and passive oscillator states. A positive diagnostic norm combines cell-volume-weighted field squares with each material term

$$\frac{|Q_j/\Delta t|^2 + \omega_{0j}^2 |P_j|^2}{A_j}. \quad (16)$$

The field term weights electric components by their instantaneous relative permittivity. This is a stopping heuristic, not a conserved discrete electromagnetic energy. CPML memories are checked for non-finite values rather than included as positive stored energy. A single displayed field slice cannot replace the complete-domain check.

Decay checks become eligible only after every finite source has finished its specified tail or sampled table, plus any extra waiting time. A continuous source disables decay termination. The observed maximum norm is frozen at the first eligible check, and the norm ratio must stay below the requested threshold for several consecutive checks. Non-finite state, an optional absolute field ceiling and a post-source growth limit report failure. Successful decay termination is distinct from user cancellation, so batch objectives still evaluate normally.

On CUDA systems with CuPy, two read-only kernels reduce array segments and then combine their partial sums. Values are promoted before squaring, including complex and weak float32 fields. This avoids full-domain double-precision copies and many small tensor launches. The NumPy and Torch implementations provide reference paths. Host diagnostic-call time includes synchronization with preceding GPU work and is not an isolated kernel timing. A full-duration checked and unchecked comparison measures actual overhead.

A truncated current pulse with nonzero time integral can leave electrostatic charge and prevent a strict decay condition. Tightening or loosening termination cannot correct the source definition. Automatic termination also does not establish Fourier convergence, especially for long-lived resonances. A longer fixed-duration calculation remains an independent temporal check.

For a reference calculation, automatic refinements can be frozen into manual boxes before removing the structure. This keeps the sample and reference node arrays identical. Mesh-dependent response changes must not be mistaken for material transmission. Exact axis nodes are retained in NPZ output. Browser field snapshots are resampled for physical-coordinate display and should not be used as a substitute for those arrays.

7 Sources and frequency-domain observables

7.1 Excitation and point traces

Electric and magnetic soft sources can be point-like or sheet-like. A sheet radiates in both directions and is not a one-way plane-wave source. Supported time waveforms include Gaussian pulses, continuous excitation, automatic broadband definitions and sampled time/amplitude/phase tables. Global source inheritance preserves local geometry, orientation and amplitude controls. Source amplitudes are reduced field increments and are not calibrated dipole moments or SI field strengths.

Cartesian excitation selects one E or H component. A vector source selects the field family and a unit direction

$$\mathbf{u} = (\sin \theta \cos \phi, \sin \theta \sin \phi, \cos \theta). \quad (17)$$

Theta is measured from the positive z axis and phi from positive x toward positive y. Each component receives the shared waveform multiplied by its signed direction weight. Exact cardinal angles have exact zero components, while intentionally weak components at other angles are retained. Separate Cartesian sources with different phases can represent elliptical excitation. One theta/phi source has one common phase.

The electric increment is applied after the E update at time $n\Delta t$. The magnetic increment is applied after the H update at $(n + 1/2)\Delta t$. Both precede monitor recording. Each vector component uses its own Yee coordinates for nearest-site placement, sheet support and Bloch phase. One requested CAD position therefore remains represented on staggered sites, which requires mesh convergence for near-field source observables. The CPU, CUDA graph, real fused CUDA and tensor-cohort paths share these conventions.

Point monitors record one selected field component at every time step. An FFT amplitude convention is retained for legacy plots. Explicit-frequency DFT uses

$$\widehat{F}(f) = \Delta t \sum_n w(t_n) F(t_n) \exp(2\pi i f t_n). \quad (18)$$

Electric and magnetic traces use their actual sample times. None, start, end and full Gaussian apodization are available. Hann is retained for point FFT workflows. Windowed field integrals are not automatically normalized powers. Uniform frequency, uniform wavelength, explicitly supplied tables, interior Chebyshev roots and Lobatto endpoints are supported. The sampling definitions and inherited settings are specified below.

7.2 Normal-incidence one-way planes

A separate source option applies paired electric and magnetic corrections at a planar total-field/scattered-field interface. This is an application of the equivalent-current principle [7]. The current implementation supports all active Cartesian propagation axes, either direction and transverse electric polarization. The plane fills a periodic transverse unit cell. Its neighboring samples must have the nondispersive background index and uniform fine spacing. Graded cells and dispersive structures are permitted away from the interface.

The incident fields are prepared on a scalar one-dimensional Yee line with the same timestep, fine mesh and background index n_b . The selected soft electric waveform drives this line eight cells before the probe. With L CPML cells at each end, the line contains $2L + 64$ cells. The default is $L = 96$, with a configurable range from 32 to 512. For propagation in the positive normal direction, total-field E samples start at index k . After the E update and before the H update, the electric correction is

$$\delta E_k^{n+1} = \frac{S}{n_b^2} h_{k-1}^{\text{inc}, n+1/2}, \quad \delta H_{k-1}^{n+3/2} = S e_k^{\text{inc}, n+1}, \quad (19)$$

where $S = c\Delta t/h$ and scalar H is oriented along the normal crossed with the electric polarization. The magnetic correction is applied after the H update. Negative propagation mirrors the interface and reverses the magnetic orientation. The prepared arrays are shared by the CPU, CUDA and tensor-cohort source paths.

The eight-cell delay and incident-line numerical dispersion are retained. Amplitude scales the auxiliary drive rather than prescribing unit incident E or calibrated power. Finite auxiliary CPML introduces residual source error, so the incident PML size requires convergence checks for weak scattering. The side toward propagation stores total fields, while the opposite side suppresses the incident wave. Oblique incidence, finite apertures and mode selection are not implemented by this option. Recognized three-dimensional normal-incidence FSP plane records have a native mapping, with general source conventions remaining open.

7.3 Closed total-field/scattered-field boxes

A closed box extends the equivalent-current construction to isolated scatterers. Let M_E and M_H select samples inside the box at their respective Yee coordinates. The required corrections to the ordinary field update are

$$\delta E = \frac{S}{n_b^2} [M_E C^-(H_i) - C^-(M_H H_i)], \quad (20)$$

$$\delta H = -S [M_H C^+(E_i) - C^+(M_E E_i)]. \quad (21)$$

The incident fields E_i, H_i solve the homogeneous discrete problem. The mask differences vanish away from the box faces. The implementation therefore stores sparse correction indices and weights with

surface-sized memory, combining corner contributions before execution. Samples inside the staggered box contain total fields and samples outside contain scattered fields.

For normal incidence, a live scalar Yee line contains $2L + 64 + N_a$ cells, where N_a is the box span along propagation in cells. It retains the eight-cell drive-to-entry delay and uses the same fine spacing, timestep and background index as the main grid. Unlike a time-by-surface table, its auxiliary field storage grows with the normal span and PML thickness. A separate scalar drive table grows with the step count. The source requires a homogeneous nondispersive shell, uniform fine cells around the box, and exterior PML on every active axis. Dispersive scatterers may lie strictly inside and graded cells may lie outside the refined neighborhood. Oblique incidence and layered incident media remain unsupported. Recognized three-dimensional normal-incidence FSP TFSF records have a native mapping, with general source conventions remaining open.

The fused CUDA E phase advances the auxiliary E field and corrects the main E faces in one launch, with both operations reading the previous auxiliary H. The H phase similarly reads the new auxiliary E. There is no cross-block dependence on an array being updated in the same launch. Separate source ordinals preserve ordering for overlapping boxes, while independent cases share the second launch dimension. Graph reset clears all auxiliary fields and CPML memory. A positive area-weighted norm of the complete incident line participates in decay diagnostics, preventing termination while an incident pulse remains in transit.

7.4 Plane monitors and Poynting flux

A frequency plane monitor can record selected electric, magnetic and Poynting components, together with signed flux. The default retains all six field components. For a 3D plane, transverse integration bins span the requested physical rectangle. For 2D, the line integral represents power per invariant length. Independent transverse strides merge integration bins while preserving their total measure. Every required component is interpolated from its own Yee locations onto the common quadrature points, including Bloch phase when an interpolation stencil crosses a cyclic seam. The normal coordinate can be the specified position or the nearest native mesh node.

The electric DFT samples use $t = (q + 1)\Delta t$. The corresponding magnetic DFT adds a half-step phase factor $\exp(i\pi f \Delta t)$. Separate apodization weights use the actual E and H times. Online accumulators retain complex values for each frequency, point and component. They are included in CUDA Graph reset state, so warm-up or capture does not contaminate the recorded spectrum.

After transferring to host complex128 arrays, the code evaluates

$$\begin{aligned} \mathbf{S}(f) &= \frac{1}{2} \operatorname{Re} \left[\widehat{\mathbf{E}}(f) \times \widehat{\mathbf{H}}(f)^* \right], \\ \Phi(f) &= \sum_j A_j \mathbf{S}_j(f) \cdot \hat{\mathbf{n}}. \end{aligned} \tag{22}$$

Host promotion avoids underflow when products of femtosecond-scale DFT integrals are multiplied by square-micrometre areas in float32. The raw quantity is reduced spectral flux. It is not a calibrated watt value. The monitor normal is the positive coordinate direction, so a reflected wave can have negative signed flux.

The accumulator channels are the dependency union of requested outputs. A single stored field requires one channel. Flux along x requires E_y, E_z, H_y, H_z , even when no fields are exported. The analogous cyclic subsets determine flux along the other axes. This reduces a flux-only plane from six to four complex channels without changing its integration points or timestamps. Stored field and Poynting arrays carry explicit component names. Omitted channels have empty axes and absent flux is represented separately, so missing data cannot be mistaken for a measured zero.

An optional complex128 accumulator retains float32 field evolution. Spatial interpolation and multiplication by the apodization weight still use field precision. It improves accumulator precision only

and does not recover field accuracy lost during propagation. CUDA planes with different accumulator precisions form separate launch groups.

For a temporal stride s , the retained indices are $q = 0, s, 2s, \dots$ and the integration weight becomes $s\Delta t$. Electric and magnetic timestamps remain $(q + 1)\Delta t$ and $(q + 1.5)\Delta t$. The CUDA observer skips gather and accumulation at other steps, while field evolution and point traces remain unchanged. Requested frequencies are checked against $1/(2s\Delta t)$. No antialias filter is applied, so the actual signal must also have negligible content above that limit. This decimation changes quadrature and is excluded from the exact-output comparisons below.

Frequency sampling supports uniform frequency, uniform wavelength, custom positive increasing tables and two Chebyshev conventions. Interior roots use $u_i = [1 - \cos(\pi(i + 1/2)/N)]/2$. Lobatto nodes use $u_i = [1 - \cos(\pi i/(N - 1))]/2$ for $N > 1$, with a midpoint when $N = 1$. The interval is interpolated in the chosen frequency or wavelength coordinate. Global frequency settings can be inherited independently of local apodization. Source-bound intervals use the union of explicitly ranged enabled sources. Other pulse definitions require explicit monitor limits.

7.5 Reference normalization

Normalized transmission divides sample flux by the absolute flux of a matching reference. Reflection first subtracts the complex reference E and H fields, then evaluates the scattered-field Poynting flux. Subtracting two already integrated powers would retain interference terms and is not equivalent.

A run signature checks region settings, source definitions, completed time-step count and exact nodes. Monitor coordinates, weights, frequency samples, temporal strides and normals must also match. Apodized normalization is rejected. By default, frequencies with reference flux below one percent of the reference maximum are invalid and return NaN ratios. Users must preserve monitor IDs and freeze automatic graded refinements when constructing an air reference. Flux-only results support direct ratios. Incident-field subtraction requires storing the four tangential fields in both runs and rejects missing channels.

This method supports controlled surface-flux ratios. It does not automatically establish a mode-resolved scattering matrix, a closed-surface absorption result, a far-field radiation pattern or an absolute collection efficiency. Those quantities require additional operators and validation.

8 CUDA execution and independent simulation ensembles

8.1 One simulation

All field, CPML and material-state arrays remain on the selected GPU during time stepping. Sources are tabulated on the actual time grid and monitor indices are prepared before execution. A complete step can be captured in a PyTorch CUDA Graph. Three warm-up steps and graph capture are followed by an explicit reset of fields, source counters, traces, CPML memory, material state and frequency accumulators. The captured graph then replays the same tensor operations with a device-resident step counter.

Snapshots transfer only the selected plane at a bounded interval. Final E/H arrays and frequency-monitor data are transferred at completion. Timings distinguish setup from the synchronized stepping loop. End-to-end ensemble measurements additionally include geometry voxelization, graph capture, result transfer and worker communication. A stepping-loop speedup is not an end-to-end application speedup.

8.2 Batch scheduling

The inherited backend changes process-global state. Threaded concurrent simulations in the same process are therefore unsafe. BatchRunner uses spawned processes, each with its own solver state and CUDA context. `parameter_sweep` creates a Cartesian product of validated parameter assignments. BatchCase

can also hold an entirely different project, allowing heterogeneous meshes, materials and monitor layouts in one ensemble.

Before submission, the runner estimates the largest case and admits a bounded number of workers per device. A worker reservation is 1.5 times the solver memory estimate plus 768 MiB for GPU context and allocator overhead. The default aggregate GPU budget is 60% of currently free memory. Idle workers release unused allocator blocks, while their process contexts may persist across optimizer generations. A user memory cap can impose a stricter limit. This admission estimate cannot prevent another application from consuming memory later.

Multiple device IDs assign independent cases to multiple GPUs. The current tests establish single-GPU process concurrency, not multi-device performance. This method is neither single-grid MPI decomposition nor a fused tensor batch. Different CUDA contexts may serialize kernel execution on one device. Increasing the worker count can therefore reduce throughput, and the benchmark must decide the useful setting for a given problem.

Each result records its case ID, parameters, status, worker PID, device, timings and metrics. Saved NPZ data is accompanied by a checksum and a fingerprint of the project and package source. Resume requires matching fingerprints, unchanged objective version keys and valid file checksums. The optimizer can therefore reuse matching completed evaluations without silently accepting stale geometry or changed objective definitions.

8.3 Black-box inverse design

The `optimize` function implements bounded DE/rand/1/bin [8]. Three distinct population members, excluding the target, define a mutant. A binomial crossover mask includes at least one forced dimension. Trial parameters are clipped to bounds and evaluated as a parallel population. A trial replaces its target when it is no worse under the selected minimization or maximization convention.

Objectives are ordinary module-level Python callables receiving the complete simulation result. They return a finite scalar or named scalar metrics. Evaluation occurs in the worker, avoiding transfer of large field arrays merely to compute a scalar. A fixed random seed and stored generation configuration make the proposal sequence reproducible. Failed simulations and invalid objective values stop the optimization explicitly rather than masquerading as physically valid candidates.

The included cylinder example maximizes a local unnormalized spectral intensity. It illustrates orchestration and does not claim a validated collection-efficiency optimum. No gradient is differentiated through the time loop, no global-optimum guarantee is provided, and fabrication constraints beyond parameter bounds are left to the project construction. Final designs require an independent mesh/time convergence check and a physically justified figure of merit.

8.4 Fused real-field Yee and CPML kernels

The optional fused CUDA path specializes two kernels to the grid shape, dtype, boundary topology and metric. Each thread computes one cell's three output components. The electric kernel forms the backward curl from H, updates the relevant CPML slab memory and writes E. The magnetic kernel forms the forward curl from the updated E and writes H. Electric source injection follows the E launch, and magnetic injection follows the H launch. Their shared CUDA stream supplies the global ordering required by the Yee leapfrog update. This is spatial operation fusion within each update, not temporal blocking across several time steps.

Unlike the tensor reference, these kernels do not materialize full-volume derivative or curl arrays. Each CPML memory array retains only its boundary slab. Periodic seam indices and graded primal/dual metric factors enter the same derivative formulas. The existing sparse ADE correction retains its implementation. Frequency monitors have a separate optional shared CUDA path described below. Bloch complex fields remain on the tensor path. Neither the new kernels nor CUDA graph replay provide an adjoint.

Torch owns the field, coefficient and state storage. DLPack supplies zero-copy CuPy views [9], and the launches use the active Torch CUDA stream. In-memory NVRTC compilation avoids filesystem-dependent source paths. Compiler options disable fused multiply-add contraction and preserve subnormal values. An initial compiler configuration flushed weak float32 fields to zero and failed the full-field comparison. The revised configuration passes that comparison and a dedicated subnormal regression. Compilation occurs before graph capture, and module lifetimes cover all captured executions.

8.5 Shared CUDA launches for independent structures

The `run_tensor_batch` API maps independent cases to the second CUDA block-grid dimension. The first dimension indexes spatial cells. For a block width of 256, the cell and case indices are

$$i = 256 \text{ blockIdx.x} + \text{threadIdx.x}, \quad b = \text{blockIdx.y}. \quad (23)$$

A device pointer table selects each case’s fields, material coefficients and CPML memory. The electric and magnetic stencil bodies are generated by the same code as the independent fused solver. No stencil reads another case’s state. Separate shared E and H source launches assign non-overlapping source samples at the corresponding update stages. A shared point-monitor launch records each case before incrementing the common time index. ADE corrections retain per-case operations between these launches. Planar frequency accumulation can use the reference operations or the shared implementation below. CUDA graph capture resets every case’s mutable state.

This implementation supports real float32 and float64 fields, two and three dimensions, matched uniform or graded grids, periodic boundaries, face-specific CPML and passive multipole materials. A cohort requires common nodes, precision, timestep, step count and boundary topology. Geometry and material values may differ. Complex Bloch fields, overlapping source supports and per-case automatic decay termination are explicitly rejected. A numerical failure aborts the cohort, while an invalid scalar objective marks the affected result as failed.

An explicit `cohort_size` partitions an ensemble into consecutive groups of at most 64 cases. The final partial group is allowed. This controls resident device state and can improve throughput when a large batch performs poorly. It does not limit accumulated host arrays when results are retained. The differential-evolution driver can evaluate each population through these shared launches by selecting tensor execution. Process execution remains the default. Both paths use the same proposal and replacement rules. Checksum resume, process fault isolation and heterogeneous resources remain features of the process runner.

A separate `run_grouped_batch` scheduler accepts interleaved projects with different meshes, durations, boundaries or real precisions. It computes the same exact topology signature used by the tensor executor and retains first-appearance order among groups. Each group is partitioned by an explicit cohort-size cap and the native memory estimate. No padding, resampling, precision change or shortened trajectory is introduced. Groups execute successively on one GPU, while cases within each cohort share CUDA launches. Results are returned in input order, but objective callbacks follow execution order. The plan reports this distinction and the original case indices. Complex fields and per-case automatic termination are rejected. This scheduler does not add an adjoint or automatic heterogeneous routing to the differential-evolution driver.

The optional `tune_tensor_batch` routine measures complete workloads for candidate cohort sizes. One warmup precedes repeated alternating measurements. Each solve includes preparation, graph capture, time stepping and final result transfer. Source duration and physical outputs are unchanged. Hash comparisons require bitwise equality of the native fields, traces, snapshots, permittivity and frequency-plane arrays across candidates. The routine does not evaluate user objectives or save simulation results. Memory-admission rejections remain in the report, while other failures stop selection.

The selected size minimizes the observed median full-solve time. The report separately records the complete tuning cost, including warmups and output verification. Selection is explicit rather than hidden

in a simulation call. It can be reused for similar design generations, but does not guarantee the fastest size on a later workload. A later independent timing series assesses that limitation. Host-side case inspection is scheduled only at requested diagnostic and snapshot steps, preserving observation order while avoiding a scan over the complete cohort on every step.

8.6 Shared spatial interpolation and frequency accumulation

The independently selectable `cuda_monitor_kernel` retains the reference plane coordinates, quadrature weights and component-specific interpolation maps. Its first kernel computes a real sample for each point j and component c ,

$$u_{jc}^q = w_c(t_q)\Delta t \sum_{\ell=1}^K a_{j\ell c} F_c^q[I_{j\ell c}], \quad (24)$$

where $K = 4$ in two dimensions and $K = 8$ in three dimensions. Electric and magnetic components use their own apodization times. A second kernel assigns each frequency, point and component to one thread and accumulates

$$\widehat{F}_{j\ell c}^{q+1} = \widehat{F}_{j\ell c}^q + p_{jc}^q u_{jc}^q. \quad (25)$$

The electric phase is $\exp[2\pi i f(q+1)\Delta t]$. The magnetic phase includes the existing additional half-step factor. The initial implementation evaluates the double-precision exponential through Torch, followed by complex64 or complex128 conversion and magnetic phase multiplication. The current implementation adds a preceding CUDA phase kernel. One thread per frequency evaluates double-precision sine and cosine at the absolute time index, converts the result to the requested precision and applies the magnetic factor. Planes with matching frequencies and timesteps share a phase buffer with one writer per frequency. There is no recurrent phase multiplication, so phase rounding does not accumulate across steps. The previous Torch expression remains available for controlled ablation.

The second block-grid coordinate chooses a plane across one or more independent cases. Pointer tables retain separate field arrays, interpolation maps, windows and accumulators. No atomic operation or inter-case reduction is required. The additional scratch storage is one real value per required field and plane point, without a volume time history or time-by-frequency table. Metadata specifies each plane's channel count, electric-channel count and temporal stride. Every scratch value is overwritten before use, while persistent DFT accumulators participate in the usual graph reset.

The option supports real float32 and float64 fields on uniform or graded meshes with periodic or CPML boundaries. Complex Bloch fields retain the reference monitor path. Observation does not change the Yee updates or material state. The interpolation sum can have a different rounding order from Torch, so complete complex spectra are compared with a declared tolerance. Fused independent and fused cohort spectra must agree bitwise. The default monitor implementation remains the Torch reference.

8.7 Graph unrolling with observation barriers

The optional `cuda_graph_steps` execution parameter captures a single-step graph and a second graph containing up to 64 consecutive unchanged steps. Independent private graph pools permit either graph to be replayed next [2]. Both captures reset all mutable Yee, CPML, ADE, auxiliary-line, trace and DFT state. Every step retains its original source samples and monitor accumulation. This reduces host replay calls without changing the physical timestep or performing shared-memory temporal blocking.

The scheduler treats every requested snapshot, diagnostic and progress callback as an exact barrier. It replays the larger graph only when the full chunk fits before the next barrier, then uses single-step replays for a shorter remainder. Single-run decay termination therefore observes the original sequence of eligible checks and stops at the same step. Tensor execution still requires fixed-duration runs. Cancellation is polled between graph replays and can have a polling interval of the selected chunk size. Frequent host observations can prevent unrolling. The default is one step because capture time and additional graph storage may outweigh loop savings.

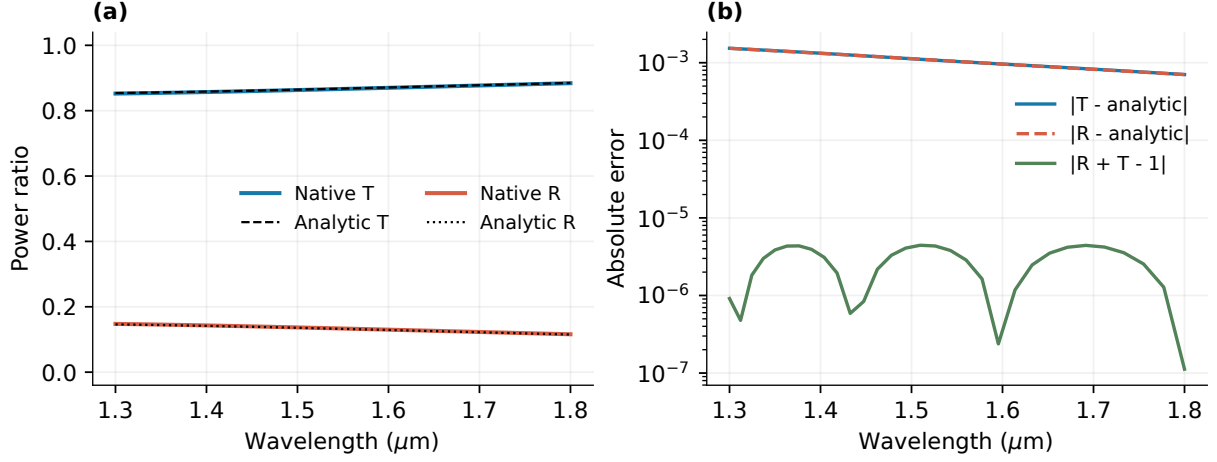


Figure 1: Native plane-monitor slab validation against the analytic lossless-slab solution. The spectra and absolute errors are plotted from the same 31 recorded frequency samples. The right axis is logarithmic.

9 Validation and reproducibility

9.1 Analytic dielectric slab

A lossless slab with refractive index 1.5 and thickness 0.2 micrometres is embedded in air. The 2D region is 8 by 0.5 micrometres with a periodic transverse boundary, a 0.025 micrometre mesh and 16 PML cells at each longitudinal end. A broadband electric sheet excites the slab, and two frequency planes measure reflection and transmission. The simulation records 1600 steps in float64. The wavelength interval is 1.3 to 1.8 micrometres at 31 uniformly spaced frequency samples.

For normal incidence, the analytic transmission is

$$T(\lambda) = \left[1 + \left(\frac{n^2 - 1}{2n} \right)^2 \sin^2(2\pi nd/\lambda) \right]^{-1}, \quad R = 1 - T. \quad (26)$$

The measured maximum absolute errors are 0.00153444 for transmission and 0.00153535 for reflection (Fig. 1). The maximum absolute residual in $R + T - 1$ is 4.43991×10^{-6} . These are absolute dimensionless errors, not percentages of small reflected powers. The errors include the staircase/interface placement and finite-grid propagation. This result validates the stated slab setup, not every possible material, source or boundary.

Synthetic tests separately exercise x-, y- and z-normal planes. They check clipped integration area, six-component interpolation and the magnetic half-step Fourier phase. Additional tests compare graded Bloch CPU and CUDA calculations in float32 and float64. The slab and synthetic tests are in `tests/test_field_monitors.py` and the executable reproduction is `python -m examples.flux_slab --backend cpu`.

9.2 Software and ensemble checks

The version 0.10 validation extends the regression suite with coupled multipole constitutive response, matched mesh studies, delayed-source termination, successful early batch completion and complete-domain diagnostic reductions. Fused kernel checks cover both precisions, 2D and 3D, asymmetric CPML, periodic seams, graded metrics, dispersive materials, graph/eager execution, subnormal fields and frequency-plane agreement. Passing these tests establishes their explicit assertions rather than exhaustive feature coverage. The dated test counts are maintained in `docs/ACCEPTANCE.md` and should be regenerated for a release.

Batch tests verify overlapping execution intervals from two distinct worker processes, equality with sequential field traces, saved-result round trips, cancellation, invalid-objective handling, memory-admission

rejection and resume fingerprint checks. A small physical source-amplitude inverse problem verifies deterministic proposal replay and improvement of a trace-based objective. The test does not establish optimizer performance on a large topology design space.

9.3 Independent vector-source recurrence

A separate source check evolves a homogeneous periodic problem in Fourier space with the analytic difference symbols $D_a^- = 1 - \exp(-iq_a)$ and $D_a^+ = \exp(iq_a) - 1$. The reference forms the curls by vector cross products and constructs the Gaussian waveform directly, without calling the native curl or waveform code. Four float64 cases cover electric and magnetic vector sources in two and three dimensions for 80 steps. The largest full-field relative L_2 difference is 2.94×10^{-15} , and the largest absolute difference is 2.50×10^{-16} in reduced units. This validates the stated discrete recurrence rather than continuum dipole radiation power. Inputs and errors are in docs/validation/vector-sources.json, reproduced by `python -m benchmarks.vector_sources`.

Regression tests additionally verify signed linear superposition, half-step previews, exact cardinal and intentionally weak components, complex Bloch magnetic sheets, planar DFT, graded CPML and tensor-cohort independence. Both field families preserve CPU/CUDA agreement in float32 and float64. Synthetic layout records verify electric theta/phi conversion while preserving the original bytes. Magnetic source-type binary mapping and calibrated source moments remain unresolved.

9.4 One-way plane checks

Twenty homogeneous float64 cases cover the active axes in two and three dimensions, both directions and both transverse polarizations. An independent scalar Yee recurrence places its boundaries beyond the discrete light cone and constructs the Gaussian without native waveform code. At 120 steps, the maximum relative full-field L_2 difference is 4.46×10^{-9} . The maximum scattered-side electric peak divided by the domain peak is 1.17×10^{-9} . These early propagation checks do not establish a long-time reflection floor.

Separate 1600-step incident-line comparisons use background indices 1, 1.5 and 3. At 96 auxiliary PML cells, the maximum correction relative L_2 difference from the long-line reference is 4.54×10^{-7} . Increasing the layer count to 192 reduces this maximum to 8.75×10^{-9} . These recorded errors isolate the finite incident-line absorption. The inputs are in docs/validation/oneway-sources.json, reproduced by `python -m benchmarks.oneway_sources`.

On RTX 5880, the index-1.5 slab test is repeated in both propagation directions with the one-way source. Across 31 wavelengths, the maximum absolute R/T error is 0.001536 and the maximum energy residual is 5.09×10^{-6} . Reflected power measured on the scattered side differs from matched-reference field subtraction by at most 2.91×10^{-6} . The corresponding empty scattered-side flux is at most 1.46×10^{-11} of the incident reference flux. These tests are reproduced by `python -m examples.oneway_slab` and recorded in docs/validation/oneway-slab.json. Additional regressions cover graded grids, transverse vectors, temporal modes, source inheritance, delayed-source termination, both precisions, graph/eager execution and tensor-cohort independence.

9.5 Closed-box and sphere-scattering checks

Twenty closed-box cases cover every active axis, both directions and transverse Cartesian polarizations in two and three dimensions. An independent scalar recurrence with ends beyond the discrete light cone supplies the reference. Component-wise physical Yee coordinates define the reference masks without calling the native surface-map or curl routines. At 90 steps, the maximum float64 full-field relative L_2 difference is 4.28×10^{-7} and the maximum exterior-to-domain peak ratio is 5.04×10^{-13} . A separate 1600-step test compares entry E/H and exit E histories for background indices 1, 1.5 and 3. Increasing auxiliary CPML from 32 to 96 and 192 cells reduces the worst relative L_2 difference from 2.13×10^{-4} to

Table 1: Closed-box TFSF sphere scattering on RTX 5880. The maximum relative cross-section error is taken over nine wavelengths. Physical domain and PML thickness stay fixed. The final row doubles the physical duration. The error sequence is not monotonic.

h (μm)	Grid	Time (fs)	Max error (%)	Empty box (μm^2)
0.1	32^3	120	8.8963	6.68×10^{-15}
0.05	64^3	120	0.3086	1.11×10^{-14}
0.025	128^3	120	1.0474	2.19×10^{-15}
0.02	160^3	120	0.5514	1.37×10^{-14}
0.025	128^3	240	1.0474	2.32×10^{-15}

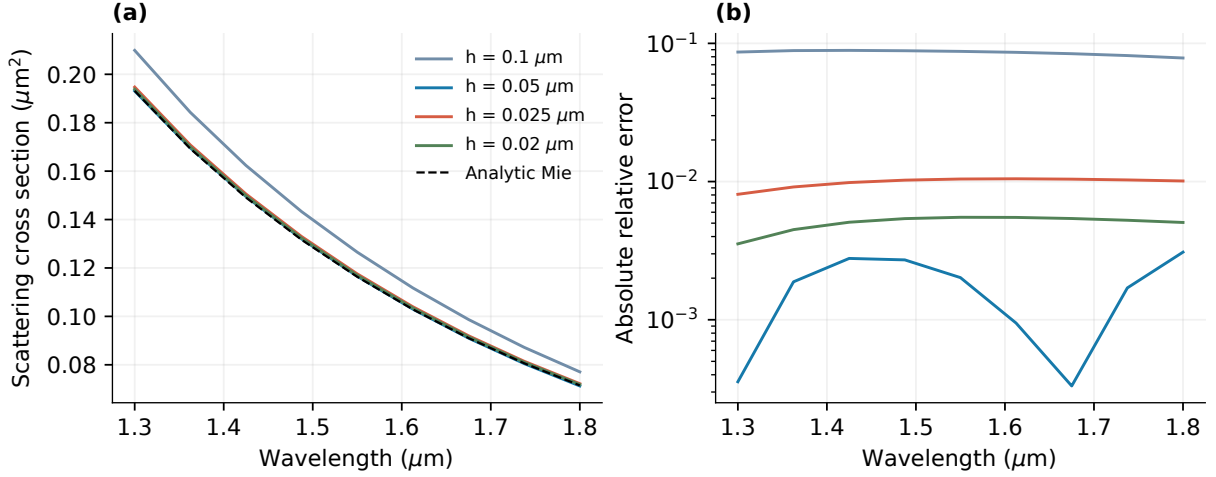


Figure 2: Closed TFSF sphere scattering. The four meshes retain the same physical domain, PML thickness and requested 120 fs duration. The analytic cross section is shown with the measured spectra in (a), and absolute relative errors in (b). The mesh error is not monotonic.

1.04×10^{-6} and 6.58×10^{-8} . These results distinguish face-correction consistency from finite incident-line error.

The scattering test uses a radius-0.3 micrometre sphere of index 1.5 in vacuum, a 3.2 micrometre cubic domain, a 1.6 micrometre TFSF box and 0.4 micrometres of physical PML per face. Six exterior planes at ± 1.1 micrometres integrate outward scattered flux. A matched empty-box run supplies incident intensity and residual-field subtraction. Nine wavelengths span 1.3 to 1.8 micrometres. The analytic scattering cross section is

$$\sigma_{\text{sca}} = \frac{2\pi}{k^2} \sum_{\ell=1}^{\ell_{\text{max}}} (2\ell + 1) (|a_{\ell}|^2 + |b_{\ell}|^2), \quad (27)$$

where the standard nonmagnetic Mie coefficients are evaluated directly from SciPy spherical Bessel functions [10]. Zero index contrast and the small-sphere Rayleigh limit provide separate checks of this reference. The direct evaluation is limited to the modest real size parameters of this fixture and is not a general robust Mie solver.

Table 1 and Fig. 2 retain the non-monotonic mesh errors. The unusually close 0.05 micrometre result does not establish a general accuracy advantage or an asymptotic convergence rate. Doubling the 0.025 micrometre duration to 240 fs changes the cross section by less than 6.5×10^{-9} relative. This isolates the duration effect for that pair, without separating geometry sampling, monitor interpolation, numerical dispersion or physical-PML errors. A requested 0.0125 micrometre mesh exceeded the existing eight-million-cell job limit and was rejected before solving. It provides no measured point. The

Table 2: Independent four-case sphere ensembles. Medians of three measured batches after warm-up, including setup, transfers, worker communication and allocator cleanup. Initial process startup is excluded. The CPU baseline is the native NumPy implementation.

Backend	Workers	Batch wall time (s)	Cases/s	Speedup vs NumPy CPU
CPU	1	47.0377	0.0850	1.00×
CUDA	1	1.1749	3.4044	40.03×
CUDA	2	1.1819	3.3845	39.80×
CUDA	4	1.2040	3.3222	39.07×

complete inputs and errors are retained in the docs/validation/tfsf-*.json records and reproduced by `python -m examples.tfsf_sphere`.

9.6 Mesh and termination studies

The convergence example places the same index-1.5 slab in an 8 by 0.8 micrometre domain, retaining 0.4 micrometres of PML thickness. The mesh spacings 0.05, 0.025 and 0.0125 micrometres use 800, 1600 and 3200 steps, respectively, to hold the physical duration fixed. The maximum transmission errors against the analytic slab decrease from 0.0062361 to 0.0015344 and 0.0003822. Both successive comparisons meet the specified absolute tolerance of 0.006. This native example is reproduced by `python -m examples.mesh_convergence` and recorded in docs/validation/mesh-convergence.json.

For termination, a separate 64^3 sphere test uses a sampled current obtained by differencing a finite dipole pulse, so its discrete time integral is zero. All timed modes use the same waveform, fused CUDA graph, 31 point-DFT frequencies and a maximum of 4000 steps. On RTX 5880, medians of three warmed repetitions are 0.099369 seconds without periodic diagnostics, 0.110668 seconds with diagnostics every 50 steps, and 0.047473 seconds with automatic decay. The threshold is 10^{-7} with three consecutive eligible checks, and the run stops at 1000 steps. Diagnostics add 11.37 percent to fixed-duration wall time, while automatic decay gives a 2.09× full-wall speedup against the unchecked full solve in this case.

The checked and unchecked full-duration final fields agree bitwise. The stopped trace matches the complete-run prefix bitwise and the relative L_2 difference of the point DFT is 1.8267×10^{-8} . The original truncated Gaussian dipole is retained as a separate residual-charge case. Its norm ratio stays near 1.3991×10^{-7} and correctly reaches the step limit without satisfying the same threshold. These cases, timing definitions and raw observations are recorded in docs/validation/run-control.json and reproduced by `python -m benchmarks.run_control_validation`. They do not establish a speedup for every source or geometry.

9.7 RTX 5880 ensemble measurements

The independent benchmark constructs four dielectric-sphere projects directly from native Python, without a commercial solver or imported commercial output. All cases use a 64^3 mesh, 800 steps, float32, physical Yee sampling, a soft dipole source and one point monitor. The sphere radius varies from 0.40 to 0.55 micrometres. `python -m benchmarks.batch_validation` compares a CPU worker against one, two and four CUDA workers after a warm batch, using three measured repetitions.

The benchmark JSON and generated table report total batch wall time, case throughput, setup and stepping time, process intervals and relative trace-energy differences. They include allocator cleanup and IPC, and exclude initial process startup from the measured repetitions. Table 2 and Fig. 3 summarize the completed measurements. No speedup is inferred from GPU specifications or the existence of concurrent processes.

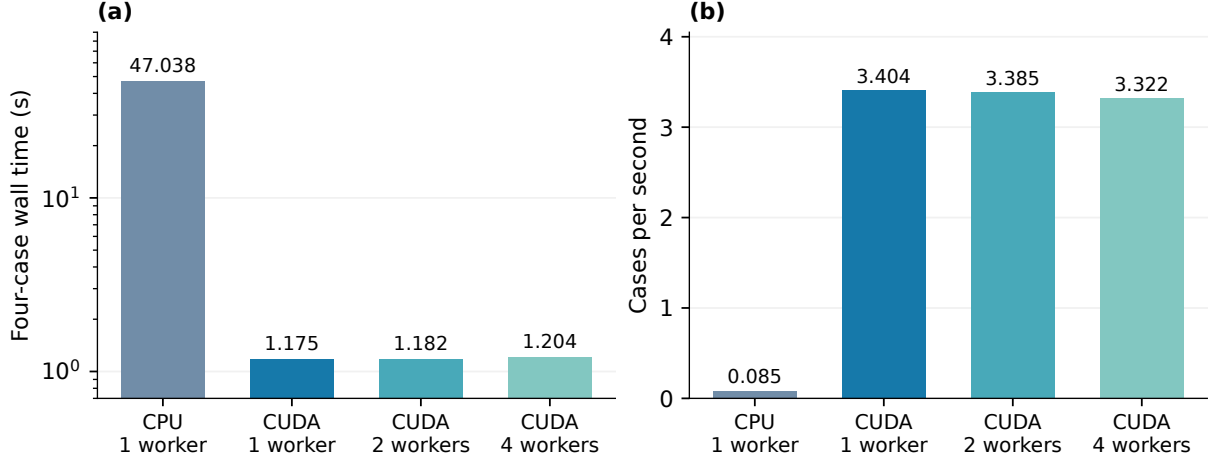


Figure 3: Total batch wall time and throughput on the RTX 5880. The CPU baseline is NumPy. Bars show medians from three measured batches after warm-up. The left axis is logarithmic. Each batch contains four independent structures.

Table 3: Paired native CUDA implementations on RTX 5880. Each value is a median over three measured runs after warm-up. The reference uses the existing PyTorch CUDA graph. Full wall time includes setup and final transfers but excludes initial context creation and compilation.

Grid	Reference loop (ms)	Fused loop (ms)	Loop speedup	Full-wall speedup
64^3	196.7	15.0	13.08×	5.76×
96^3	286.8	32.7	8.77×	3.90×
128^3	748.7	123.5	6.06×	3.67×

The one-worker CUDA batch is fastest for this case. Two and four workers do not improve throughput. The CPU baseline is the same native NumPy implementation, not a compiled commercial solver. The largest relative trace-energy difference against CPU is 0.

9.8 Independent validation limits

Validation is based on mathematical identities, analytic optical solutions and independently authored native CPU/GPU projects. Reproduction scripts, input models and raw timing records accompany the measurements.

The stated tests cover specific geometries and conditions. Future work must establish convergence for curved interfaces, resonant dispersive media, grazing incidence and physically calibrated observables. No broad commercial-solver equivalence or universal mesh superiority follows from the native slab and sphere benchmarks.

9.9 Fused CUDA measurements on RTX 5880

A separate benchmark compares the two native CUDA implementations using a dielectric sphere of index 1.444 and radius 0.5 micrometres in a 4.8 micrometre cubic domain. Both paths use float32, physical Yee material sampling, eight CPML layers on each face, a Gaussian soft dipole, one point trace and a final snapshot. Each mesh is run for 800 steps. One warmup precedes three measured repetitions with alternating implementation order. The full simulation wall time includes voxelization, graph preparation, the time loop and final data transfer. Initial context and compilation costs are excluded by warmup.

Table 4: Eight forward fixtures on RTX 5880. The external baseline is flaport/fdtd 0.2.2 with an added CUDA Graph adapter. Values are medians of three warmed full solves. The last column is point-trace relative L_2 difference in percent, not analytic error. The PML interface stencils differ and final fields are not identical.

Fixture, grid	Upstream graph (ms)	Native fused (ms)	Speedup	Trace L_2 (%)
Vacuum, 64^3	649.23	38.06	17.06×	0.0142
Sphere, 64^3	629.07	39.72	15.84×	0.0233
Slab, 64^3	636.26	37.23	17.09×	0.0197
Waveguide, 64^3	632.59	39.90	15.85×	0.0372
Vacuum, 96^3	954.98	78.77	12.12×	0.0120
Sphere, 96^3	848.58	88.05	9.64×	0.0208
Slab, 96^3	836.66	80.37	10.41×	0.0174
Waveguide, 96^3	848.07	80.00	10.60×	0.0328

Table 3 summarizes the paired measurements. The final E and H arrays and complete point traces agree bitwise in all three tests. Torch peak allocated memory decreases by approximately 25 to 27 percent. That metric excludes driver and context allocations. The physical timestep varies with mesh spacing, so this is a paired implementation benchmark at each size rather than a mesh convergence study. The reference is PhotonWeave’s existing tensor CUDA graph, and these figures establish no performance ranking against another library. Raw data is in docs/validation/cuda-kernels.json and the executable benchmark is `python -m benchmarks.cuda_kernels`.

9.10 External solver and tensor-cohort measurements

The external comparison uses the installed flaport/fdtd 0.2.2 grid and its unmodified electric and magnetic updates. Both engines receive identical component-sampled permittivity, timestep, point-source samples and point-monitor location. Our adapter also captures the upstream operations in a CUDA Graph. The resulting upstream graph fields and traces agree bitwise with its eager execution. This avoids treating eager host dispatch as the only external baseline.

The fixtures are vacuum, a dielectric sphere, a slab and a point-driven dielectric guide in a 4.8 micrometre cubic domain. The dielectric index is 1.5. The sphere radius is 0.5 micrometres, the slab thickness is 0.3 micrometres and the guide cross-section is 0.6 by 0.4 micrometres. Each fixture uses 64^3 and 96^3 meshes, 800 steps, float32, eight PML cells per face and a Gaussian point source with central wavelength 1.55 micrometres. Automatic termination and periodic divergence checks are disabled in every timed mode. Three alternating repetitions follow a warmup. Full wall time includes construction, graph preparation and complete final field transfer, while first compilation and context startup are excluded.

The full-wall speedups in Table 4 range from 9.64 to 17.09 against the graph-adapted upstream baseline. Point-trace relative L_2 differences are 0.0120 to 0.0372 percent, below the predeclared one-percent threshold. The upstream high-side electric PML interface omits a derivative included by the native stencil, so the solvers are not discretely identical. In the late-time 64^3 vacuum case, final H has a relative L_2 difference of 0.314542, a maximum absolute difference of 1.88582×10^{-7} and a reference peak of 1.22583×10^{-7} . These full-field differences are retained in the record. Point-trace agreement alone does not establish equivalent modal transmission, resonant quality factor or absorption accuracy.

The tensor-cohort benchmark varies the sphere radius from 0.35 micrometres in increments of 0.01 micrometres. It compares native independent fused solves with cohorts containing 1, 2, 4, 8 and 16 structures on 32^3 and 64^3 grids. Every measured native final E/H array and point trace agrees bitwise. Table 5 includes both improvements and regressions. For the trace-peak objective, cases per second is also objective evaluations per second. The 32^3 , 16-case cohort is 2.40 times faster than independent native execution, while the large 64^3 cohorts are slower.

Table 5: Native sphere ensembles on RTX 5880 with a real CUDA batch axis. Full wall times include setup, transfers and scalar objective evaluation. Final E/H arrays and point traces match independent native runs bitwise. Values below unity in the last column are slowdowns.

Grid, cases	Sequential (ms)	Tensor cohort (ms)	Cases/s	Speedup
32^3 , 1	23.22	20.38	49.06	1.14×
32^3 , 2	46.00	31.39	63.71	1.47×
32^3 , 4	106.81	63.67	62.83	1.68×
32^3 , 8	181.89	88.24	90.66	2.06×
32^3 , 16	341.29	142.46	112.31	2.40×
64^3 , 1	39.08	40.49	24.70	0.97×
64^3 , 2	86.40	78.29	25.55	1.10×
64^3 , 4	167.68	153.87	26.00	1.09×
64^3 , 8	307.35	357.10	22.40	0.86×
64^3 , 16	669.53	852.94	18.76	0.78×

A follow-up 64^3 , 16-case experiment measures explicit splitting. Median full wall times are 0.667887 seconds for native sequential execution, 0.607879 seconds for four cohorts of four and 0.866730 seconds for one cohort of sixteen. Peak Torch allocation decreases from 240.61 MiB for the single large cohort to 62.40 MiB for the split execution, excluding driver and context memory. Splitting preserves bitwise fields and traces. These results motivate measured cohort selection, but they do not establish an optimal general policy or identify the hardware bottleneck without profiling.

The records are docs/validation/open-source-flaport.json, docs/validation/tensor-batch.json and docs/validation/cohorts.json. Their scripts retain exact project settings, all timing repetitions, errors and objective values. The installed upstream grid and boundary modules are fingerprinted. FDTDx, ftdtz and ftdtd3d have not been timed on this GPU. A compatible environment and validated adapters remain necessary before comparing their speed. No cross-library leadership claim follows from this single-library comparison.

9.11 Cohort selection and complete design loops

A follow-up evaluates 16-case ensembles for vacuum source amplitudes, sphere radii, slab thicknesses and waveguide widths, on both 32^3 and 64^3 grids. Every solve uses 800 steps, float32 and the same point-source and point-monitor workload. The external graph adapter evaluates cases sequentially, whereas native cohorts share CUDA launches. These workflow throughput ratios do not describe an upstream fused-batch implementation. All native final E/H arrays and traces agree bitwise with independent native solves. The maximum external point-trace relative L2 difference is 0.3893 percent, below the predeclared 1 percent gate. Final-field differences remain in the raw record and are not asserted to meet that gate.

Before the reported timing series, the selector measures sizes 1, 2, 4, 8 and 16 using one warmup and three trials each. The later runs are independent of those selection samples. Their native throughput improvements range from 1.07 to 2.39 relative to independent native execution. Tuning costs 5.51 to 16.65 seconds for these examples. Dividing that cost by the later measured saving predicts 30 to 420 repeated identical ensembles to repay selection, assuming stable timings. That estimate is not a measurement of future workloads. In the 32^3 vacuum case, the selected size of two is slower on the later series than a fixed size of sixteen, with a throughput ratio of 0.80. All candidate samples and this regression are retained. Three timing repetitions do not establish confidence intervals or a universally optimal choice.

The complete design benchmark uses DE/rand/1/bin with seed 73, population 16 and three trial generations after initialization. Each loop therefore evaluates 64 forward simulations. The design variable is sphere radius in the interval $[0.25, 0.75]$ micrometres and the objective is integrated squared point-field

Table 6: Sixteen-case ensembles on RTX 5880 after a separate cohort-selection experiment. The external baseline runs graph-adapted flaport/fdtd cases sequentially. Native execution shares CUDA launches. These are workflow throughput ratios, not single-solve or upstream fused-batch comparisons. Tuning cost is excluded here and reported separately in the raw record and text.

Fixture, grid	Cohort	Upstream (s)	Native batch (s)	Speedup
Vacuum, 32^3	2	6.523	0.209	31.20×
Sphere, 32^3	16	6.665	0.207	32.22×
Slab, 32^3	16	6.977	0.204	34.12×
Waveguide, 32^3	16	6.436	0.145	44.32×
Vacuum, 64^3	4	7.911	0.519	15.24×
Sphere, 64^3	4	8.290	0.545	15.22×
Slab, 64^3	4	7.949	0.503	15.81×
Waveguide, 64^3	4	8.058	0.562	14.33×

Table 7: Complete seeded differential-evolution loops on RTX 5880. Each loop includes 64 forward solves and the scalar objective. Three warmed repetitions are summarized by their medians. Fixed cohorts contain 16 cases at 32^3 and four cases at 64^3 . Full optimization histories are identical between modes. There is no adjoint.

Grid	Independent (s)	Tensor (s)	Speedup	Tensor evaluations/s
32^3	1.677	0.818	2.05×	78.28
64^3	2.468	2.204	1.12×	29.04

amplitude. The native independent and tensor paths produce identical parameter and objective histories. Table 7 reports complete-loop costs, including proposal generation, full result transfer and objective evaluation. The fixed sizes are sixteen at 32^3 and four at 64^3 , without tuning in these timings. This is a small forward-only black-box problem, not gradient-based topology optimization or a proof of a global optimum.

The ensemble and design records are `docs/validation/ensembles.json` and `docs/validation/design-throughput.json`. Their executable modules are `benchmarks.ensemble_comparison` and `benchmarks.design_throughput`. A separately collected before-change sphere record is retained, but it does not isolate the effect of the host observation scheduler. No isolated scheduler speedup is inferred from those unpaired measurements.

9.12 Spatial-spectral ensemble ablation

A separate study adds three spatial planes with all six complex E/H components at nine frequencies, without spatial downsampling. It repeats vacuum, sphere, slab and waveguide ensembles on 32^3 and 64^3 grids. Each row contains eight 800-step float32 solves with cohorts of four. All six execution modes receive one warmup followed by three measured repetitions in alternating order. Full wall time includes setup, graph preparation and output transfer. Cold context creation, first compilation and disk output are excluded. This initial record uses the Torch phase expression and is retained unchanged after the later phase-kernel revision.

The external baseline still calls unmodified flaport/fdtd 0.2.2 E/H updates through a CUDA Graph. It receives either the common Torch observation adapter or the same new fused DFT adapter used by PhotonWeave. The table uses the latter baseline, while the raw record retains both. This is an explicit observation adapter, not a claim that upstream exposes a spatial DFT or independent-cohort API. Native sequential and batch modes independently select Torch or fused DFT, holding the fused Yee updates fixed.

Table 8: Eight independent cases per row on RTX 5880, with three six-component spatial DFT planes, nine frequencies, 800 steps and float32. Values are medians of three warmed full solves. The upstream sequence uses flaport/fdtd 0.2.2 with CUDA Graph and the same new fused DFT adapter. Both native batch modes use cohorts of four and fused Yee updates. The final column isolates monitor fusion at that fixed cohort size.

Fixture, grid	Upstream sequence (s)	Torch DFT batch (s)	Fused DFT batch (s)	Monitor speedup
Vacuum, 32^3	3.444	1.367	0.182	7.50×
Sphere, 32^3	3.458	1.434	0.204	7.01×
Slab, 32^3	3.465	1.325	0.163	8.12×
Waveguide, 32^3	3.364	1.331	0.181	7.35×
Vacuum, 64^3	4.111	1.560	0.370	4.22×
Sphere, 64^3	4.248	1.581	0.436	3.63×
Slab, 64^3	4.114	1.606	0.395	4.06×
Waveguide, 64^3	4.145	1.563	0.382	4.09×

At a fixed cohort size, monitor fusion improves full wall time by 3.63 to 8.12 times. Comparing the same new monitor implementation in native sequential and cohort execution gives a separate 1.07 to 1.73 times improvement. The ratio against the upstream sequence with the identical fused observer is 9.75 to 21.23. These are ensemble workflow ratios rather than single-solve or upstream fused-batch results. Peak allocated Torch memory rises from 2.66 to 10.61 MiB for sequential versus cohort execution at 32^3 , and from 17.92 to 71.63 MiB at 64^3 . Driver and context allocations are excluded.

Every native final E/H array and point trace matches bitwise across monitor choices. Fused independent and cohort complex spectra also agree bitwise. The maximum complete complex-plane relative L_2 difference against the native Torch observer is 5.28×10^{-8} , below the predeclared 3×10^{-6} gate. The maximum external spectral difference is 0.388 percent, below a separate 1 percent gate. External final-field differences and their absolute reference scales remain in the record without a full-field equivalence claim. These checks do not establish analytic error, mode efficiency or mesh convergence.

The independently authored projects, every timing sample, source hashes and numerical gates are retained in docs/validation/spectral-ensembles.json. The experiment is reproduced by `python -m benchmarks.spectral_ensemble`. The implementation and regression tests also cover real float32/64, nonuniform and periodic interpolation, distinct plane supports, apodization, graph/eager execution and mixed monitor selections. FDTD_X, fdtd_z and fdtd_{3d} remain unmeasured on this GPU.

9.13 Phase-kernel and graph-unrolling follow-up

The next experiment retains the same eight-case, three-plane, nine-frequency workload and compares seven modes. The external sequence and native independent and cohort executions each use one-step and eight-step graphs with the current CUDA phase kernel. A seventh mode retains the previous Torch phase expression within the native cohort, with one-step graphs and unchanged fused interpolation and accumulation. Every mode receives one warmup and three alternating measured repetitions. All capture costs enter full wall time. Table 9 uses the smaller of the two external median costs and a fixed one-step native cohort.

The resulting workflow ratios against the external sequence range from 9.87 to 21.44. Isolating phase evaluation gives loop ratios of 1.06 to 1.19, but full-wall ratios range from 0.94 to 1.06. Three of eight full-wall rows regress. Eight-step graph replay gives full-wall ratios of 0.96 to 1.09 and loop ratios of 0.93 to 1.04, also with regressions. These observations support fewer phase operations but do not establish a universal end-to-end improvement. The default graph width remains one. Torch peak reserved memory for native cohorts falls from 18 to 14 MiB at 32^3 and from 90 to 86 MiB at 64^3 with the phase revision. These allocator metrics exclude context, driver and graph-executable allocations outside Torch.

Table 9: Phase-kernel and graph-unrolling follow-up with the same eight-case spectral workload on RTX 5880. Times include preparation of all requested graphs and final transfer. The external sequence receives the same current observer, using the lower median of its one-step and eight-step options. Native batch uses cohorts of four and one-step graphs. Phase gain divides the previous phase expression cost by the new cost. Graph gain divides one-step cost by eight-step cost with the new phase kernel. Both gains use full wall time and retain values below one.

Fixture, grid	Upstream sequence (s)	Native batch (s)	Phase gain	Graph gain
Vacuum, 32^3	3.371	0.179	1.03×	1.01×
Sphere, 32^3	3.409	0.198	1.03×	0.96×
Slab, 32^3	3.613	0.171	0.95×	1.02×
Waveguide, 32^3	3.301	0.154	1.06×	0.99×
Vacuum, 64^3	4.026	0.396	0.94×	1.09×
Sphere, 64^3	4.324	0.382	1.03×	0.99×
Slab, 64^3	4.107	0.416	1.04×	0.96×
Waveguide, 64^3	4.147	0.390	0.99×	1.01×

Every current native independent, cohort and unrolled output array agrees bitwise, including final fields, traces, snapshots, permittivity and complex spectra. External unrolling also agrees bitwise with its own one-step graph. The previous and current phase expressions differ by at most 3.42×10^{-8} in complete complex DFT relative L_2 , below the predeclared 3×10^{-6} gate. The maximum external complex spectral difference remains 0.388 percent. Cross-library final fields remain different. Separate regressions check both precisions, heterogeneous observation barriers, material and auxiliary-source state, complex single-run fields, exact decay termination and callback-triggered cancellation. Absolute phase checks cover 257 frequencies and time indices above one billion without recurrent phase updates.

The 168 measured ensemble runs, full inputs, installed upstream hashes, implementation hashes and all numerical checks are retained in docs/validation/graph-ensembles.json. The executable is `python -m benchmarks.graph_ensembles`. The README and this table are generated from the retained measurements. Three repetitions do not establish confidence intervals, and the study does not add an adjoint or measurements against the other three GPU libraries.

9.14 Selective-output ensemble measurements

A further experiment requests signed flux from three planes at 65 frequencies. Each row again contains eight cases, 800 float32 steps and native cohorts of four. One mode stores all six fields, three Poynting components and flux. The selective modes accumulate the four required tangential fields internally and export flux. Neither spatial nor temporal decimation is used. The upstream sequence receives the same selective CUDA observer and both graph widths, and the comparison uses its lower measured median. Native graphs use one step. Full wall includes setup, capture and all selected result transfers.

Output selection improves native cohort full wall by 1.14 to 1.69 times across these eight rows. Torch peak allocation decreases from 18.51 to 14.88 MiB at 32^3 and from 103.15 to 96.25 MiB at 64^3 . Reserved peaks decrease from 22 to 18 MiB and from 122 to 100 MiB, respectively. These metrics exclude context, driver and graph-executable memory outside the Torch allocator. At fixed selective outputs, cohort scheduling contributes a separate 1.003 to 1.60 times improvement over native independent execution. The small 64^3 slab difference does not establish a robust scheduling gain from three samples.

The workflow ratios against the selective upstream sequence are 7.74 to 19.32. All 120 timed-ensemble gates pass. Native final E/H, point traces and signed flux agree bitwise with independent full-output runs. Maximum upstream relative L_2 differences are 0.3267 percent for point traces and 0.1314 percent for flux, below the predeclared 1 and 2 percent gates. Cross-library full-field differences remain in the raw record without a field-equivalence claim. These gains apply when the discarded fields are not requested. They do

Table 10: Selective-output ensembles on RTX 5880. Each row uses eight cases, 800 steps, float32 and three planes with 65 frequencies. Times are medians of three warmed full solves. Native cohorts contain four cases with one-step graphs. The upstream sequence uses the identical selective observer and the lower median of its one-step and eight-step graphs. Full output stores six fields and three Poynting components. Flux-only stores the same signed flux with four internal fields. Selection gain is full-output time divided by flux-only time, with no decimation.

Fixture, grid	Upstream sequence (s)	Full-output batch (s)	Flux-only batch (s)	Selection gain
Vacuum, 32^3	3.404	0.241	0.211	1.14×
Sphere, 32^3	3.381	0.220	0.175	1.26×
Slab, 32^3	3.324	0.239	0.194	1.23×
Waveguide, 32^3	3.475	0.244	0.213	1.14×
Vacuum, 64^3	4.388	0.833	0.492	1.69×
Sphere, 64^3	4.298	0.882	0.541	1.63×
Slab, 64^3	4.208	0.847	0.544	1.56×
Waveguide, 64^3	4.204	0.889	0.533	1.67×

Table 11: Native rectangular mesh ablation on RTX 5880. Each row uses four transverse-invariant cases with the same propagation step, actual time step and flux observables. The first column gives the longitudinal cell count. Times are medians of three warmed full-wall solves. Mesh gain compares uniform and rectangular batches. Batch gain compares independent and batch execution at the rectangular mesh. This is not a cross-library ranking.

Fixture, N_x	Uniform batch (s)	Rectangular batch (s)	Mesh gain	Batch gain
Vacuum, 128	0.312	0.060	5.18×	1.80×
Slab, 128	0.326	0.075	4.33×	1.62×
Bilayer, 128	0.326	0.075	4.34×	1.54×
Multilayer, 128	0.336	0.069	4.88×	1.70×
Vacuum, 192	1.175	0.088	13.39×	1.48×
Slab, 192	1.199	0.099	12.09×	1.41×
Bilayer, 192	1.203	0.095	12.67×	1.42×
Multilayer, 192	1.255	0.094	13.37×	1.46×

not establish a six-field-output speedup, mesh-converged accuracy or an advantage over the other three unmeasured libraries.

The experiment is reproduced by `python -m benchmarks.selective_monitors`. Every project, repetition, source hash and numerical check is retained in `docs/validation/selective-monitors.json`. Separate tests compare selected fields and Poynting components against independent time integrals on all three plane normals. They cover CPU and CUDA, mixed accumulator precision, spatial strides, temporal decimation, named NPZ outputs, missing-component errors and single/cohort equivalence.

9.15 Matched-time-step rectangular mesh ablation

A separate native experiment tests unnecessary transverse resolution in normal-incidence vacuum and dielectric layer stacks. Four independent cases per row vary material indices and source phase. Each case uses 800 float32 steps and records a point trace and one signed-flux plane at 17 frequencies. The propagation step is 0.05 micrometres. Both transverse steps change from 0.05 to 0.2 micrometres, removing 93.75 percent of cells. An override retains the baseline time step exactly, together with the physical domain, longitudinal PML, source and frequency grid. Every structure and excitation is invariant in the transverse directions.

Table 12: Native geometry-preparation ablation on RTX 5880. Four cases per row each contain eight solids. Both modes use identical grids, 800 float32 steps, four-case cohorts and complete outputs. Medians summarize three warmed alternating repetitions. Full wall includes host preparation, graph capture, stepping and result transfer. The last column isolates host material preparation. This is not a cross-library comparison or a new CUDA stepping kernel.

Solids, grid	Unpruned batch (s)	Bounded batch (s)	Full-wall gain	Host prep. gain
Spheres, 64^3	0.242	0.161	1.50×	6.96×
Rotated boxes, 64^3	0.608	0.185	3.28×	24.52×
Concave polygons, 64^3	3.965	0.194	20.40×	115.92×
Elliptical sectors, 64^3	2.015	0.191	10.54×	77.55×
Spheres, 96^3	0.888	0.617	1.44×	10.27×
Rotated boxes, 96^3	2.023	0.623	3.25×	41.70×
Concave polygons, 96^3	13.768	0.650	21.19×	244.47×
Elliptical sectors, 96^3	6.414	0.628	10.21×	127.52×

The two uniform grid shapes are $128 \times 64 \times 64$ and $192 \times 96 \times 96$, reduced to $128 \times 16 \times 16$ and $192 \times 24 \times 24$. Cohorts contain four cases. Three alternating measured repetitions follow a warmup. Full wall includes setup, graph capture and final E/H and monitor transfer, while excluding cold context/compilation, accuracy checks and disk writes. The mesh change improves batch full wall by 4.33 to 13.39 times. At the reduced mesh, batching adds a separate 1.41 to 1.80 times gain over independent native execution. Torch peak allocation decreases from 120.71 to 7.63 MiB and from 386.24 to 24.23 MiB, respectively. Allocations outside the Torch allocator remain excluded.

All 72 timed-ensemble gates pass. Final centerline E/H and complete point traces agree bitwise with independent uniform native runs. Every final field array is constant in the transverse directions. Maximum signed-flux relative L_2 difference is 5.95×10^{-15} , below the predeclared 3×10^{-5} gate. The grids have different sample counts, and this comparison uses their common observables. This is neither a cross-library comparison nor a curved-geometry convergence result. A scalar one-dimensional implementation could exploit more of this geometry’s invariance. The experiment only quantifies the cost of retaining unnecessary transverse cells in the native three-dimensional implementation. Inputs, source hashes, repetitions and errors are in docs/validation/rectilinear-ensembles.json. The executable entry point is `python -m benchmarks.rectilinear_ensembles`.

9.16 Analytic geometry preparation in CUDA ensembles

A native ablation evaluates material preparation for four scene families at 64^3 and 96^3 . Each ensemble has four independent cases, each containing eight compact solids. All runs use the same physical scene, Yee sampling, 800 float32 steps, CPML, point trace, six-component frequency plane and four-case CUDA cohort. The baseline evaluates the same analytic membership equations over the complete domain. The bounded version restricts those equations to each solid’s conservative support. Neither version caches a material result between runs.

Three warmed alternating repetitions give full-wall gains of 1.44 to 21.19 times for this workload. The largest gains occur for concave polygons, whose full-domain crossing tests dominate the unpruned cost. Host preparation time is recorded separately from setup, stepping and full wall. All 48 timed-ensemble checks pass bitwise for permittivity, complete final E/H, point traces, time arrays, snapshots, complex plane fields and signed flux. Torch peak allocation remains identical between modes at each grid size, since all field and observation arrays remain allocated. This allocator metric does not include host material-preparation arrays.

Table 13: Mixed-grid or mixed-duration ensembles on RTX 5880. Each row interleaves 16 cases from four geometry families. The first column gives grid sizes and time-step counts. All cases retain three six-component planes at nine frequencies. Medians summarize three warmed repetitions. Full wall includes grouping and all setup and output transfer. Upstream uses the identical fused observer and the lower median of one-step and eight-step graphs. Grouped execution uses four-case cohorts and one-step graphs. The last column compares native sequential and grouped execution.

Grid, steps	Upstream sequence (s)	Native sequence (s)	Native grouped (s)	Native gain
$32^3 + 48^3$, 800	6.958	0.649	0.455	1.43×
$32^3 + 64^3$, 800	7.892	0.722	0.578	1.25×
32^3 , 400/800	5.827	0.522	0.348	1.50×
64^3 , 400/800	6.337	0.891	0.852	1.05×

These measurements establish a native preparation improvement for compact analytic solids, not a faster CUDA update kernel or a ranking against another library. Some repetitions show different loop times despite identical kernels, and the record retains these variations without assigning a kernel-level cause. Long propagation runs and large overlapping objects may gain less. No grid reduction, subpixel correction or adjoint is introduced. The complete inputs, source hashes, repetitions and output gates are retained in `docs/validation/geometry-ensembles.json` and reproduced by `python -m benchmarks.geometry_ensembles`.

9.17 Mixed-topology ensemble scheduling

A further benchmark interleaves vacuum, sphere, slab and waveguide families across two mesh sizes or two durations. Each row contains 16 independent float32 cases. Two rows combine 32^3 with either 48^3 or 64^3 , with 800 steps per case. Two rows retain 32^3 or 64^3 and mix 400-step and 800-step trajectories. All modes use the same per-case physical and numerical inputs. Three planes retain all six complex field components at nine frequencies, together with full final E/H, point traces and native snapshots.

The grouping algorithm uses an explicit cap of four cases per cohort and one-step native graphs. Full wall includes classification, validation, preparation, graph capture, stepping and output transfer. Cold context and compilation, accuracy checks and disk output are excluded. Three alternating measured repetitions follow one warmup per mode. Grouping and preflight cost 38.2 to 39.5 milliseconds in these rows. The resulting native full-wall gain is 1.05 to 1.50 times over independent sequential execution. The smallest gain is modest, and three samples do not establish a confidence interval. Torch peak allocation increases from 8.00 to 31.98 MiB for the $32^3 + 48^3$ ensemble, from 17.92 to 71.63 MiB for ensembles containing 64^3 grids, and from 2.66 to 10.61 MiB for the mixed-duration 32^3 case. Driver, context and external graph allocations remain excluded.

The external flaport baseline receives the same fused DFT observer and both one-step and eight-step graph options. The lower external median gives workflow ratios of 7.44 to 16.76 against grouped native execution. All 48 timed-ensemble gates pass. Native complete outputs agree bitwise with independent native runs. Maximum external point-trace and complex-plane DFT relative L_2 differences are 0.3233 and 0.3875 percent, below the predeclared 1 percent gates. External final fields remain different and their errors are retained. This experiment measures existing fused cohorts reached through a new mixed-input scheduler. It does not establish a new stencil-kernel speedup, an upstream fused-batch comparison or a ranking against the three other unmeasured libraries. Exact inputs, source hashes, repetitions and checks are in `docs/validation/grouped-ensembles.json`. The entry point is `python -m benchmarks.grouped_ensembles`.

10 Limitations and development priorities

The native scope must be assessed by tested physical capabilities, supported parameter ranges and convergence evidence. Counting configuration properties cannot establish a product-completion percentage. In particular, a shared UI and Python schema does not establish that every requested observable or source type is physically implemented.

Near-term numerical priorities include oblique injection, eigenmode ports and mode expansion, validated conformal/subpixel interfaces, sampled-data dispersion fitting, additional boundary families and distributed time/volume monitors. Workflow priorities include more compact ensemble execution, a complete results browser, resource-aware graphical sweeps and checkpoint/restart. Adjoint differentiation requires a separate derivation and validation of material, source, monitor and boundary sensitivities.

The current browser uses an original identity, artwork and source code. Its workflow is organized around objects, regions, sources and monitors. The principal computational gaps are a validated discrete adjoint, low-cost predictive cohort selection, complex or independently terminating tensor cohorts and single-problem multi-GPU domain decomposition. The implemented cohort selector requires explicit timing trials. Comparisons against additional libraries require common accuracy targets and a compatible measured GPU environment before any performance-leadership claim.

11 Conclusions

PhotonWeave provides a shared Python and graphical workflow for a defined subset of time-domain photonics simulations. Its implementation exposes the numerical conventions needed to interpret staggered fields, dispersive state, nonuniform mesh metrics and normalized flux. A separate ensemble layer supports repeated simulation and bounded black-box inverse design. The software and manuscript retain explicit distinctions between implemented capability, tested accuracy and outstanding work. Broader replacement claims, general performance guarantees and public distribution readiness remain unestablished.

Data, code and authorship statement

The project source, tests and native examples are maintained in a local development workspace. A public release is not yet available. This manuscript and the associated implementation were prepared with AI assistance. Author review of the scientific content and software validation is pending before submission.

References

- [1] Floris Laporte et al. fdtd: A three-dimensional electromagnetic FDTD simulator in Python. MIT-licensed software. URL <https://github.com/flaport/fdtd>. Accessed 19 September 2026.
- [2] PyTorch contributors. CUDA semantics and CUDA Graphs. PyTorch documentation. URL <https://docs.pytorch.org/docs/2.10/notes/cuda.html>. Accessed 19 September 2026.
- [3] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.
- [4] Changhyun Kim and Byoungcho Lee. TORCWA: GPU-accelerated Fourier modal method and gradient-based optimization for metasurface design. *Computer Physics Communications*, 282:108552, 2023. doi: 10.1016/j.cpc.2022.108552. Code: <https://github.com/kch3782/torcwa>.
- [5] Kane S. Yee. Numerical solution of initial boundary value problems involving Maxwell’s equations in isotropic media. *IEEE Transactions on Antennas and Propagation*, 14(3):302–307, 1966. doi: 10.1109/TAP.1966.1138693.

- [6] J. Alan Roden and Stephen D. Gedney. Convolution PML (CPML): An efficient FDTD implementation of the CFS-PML for arbitrary media. *Microwave and Optical Technology Letters*, 27(5):334–339, 2000. URL <https://onlinelibrary.wiley.com/doi/10.1002/1098-2760%2820001205%2927%3A5%3C334%3A%3AAID-MOP14%3E3.0.CO%3B2-A>.
- [7] Ardavan Oskooi and Steven G. Johnson. Electromagnetic wave source conditions. arXiv:1301.5366, 2013. URL <https://arxiv.org/abs/1301.5366>.
- [8] Rainer Storn and Kenneth Price. Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11:341–359, 1997. doi: 10.1023/A:1008202821328.
- [9] CuPy contributors. Interoperability with PyTorch and DLPack. CuPy documentation. URL https://docs.cupy.dev/en/stable/user_guide/interoperability.html. Accessed 19 September 2026.
- [10] Scott Prahl. Mie scattering algorithms. miepython documentation. URL https://miepython.readthedocs.io/en/latest/07_algorithm.html. Accessed 19 September 2026.